Indice

1 Introduzione a Python **2**

2 Il Flusso di Esecuzione **2**
2.1 Confronto tra C e Python 3
2.2 Esempio 4
2.3 Conclusione 5

3 Trees Hierarchy **6**
3.1 General Tree 6
3.1.1 General Tree - Abstract Data Type 7
3.2 Binary Tree 7
3.2.1 Binary Tree - Abstract Data Type 8
3.2.2 Properties of Binary Trees 9
3.3 General Tree Traversal 11
3.3.1 Preorder e Postorder 11
3.3.2 Breadth-First Tree Traversal 13
3.3.3 Inorder Traversal 14
3.3.4 Tree Traversal in Python 15

Relazione sul Funzionamento Interno di Python

Appunti di Programmazione

20 marzo 2026

1 Introduzione a Python

Python è un linguaggio di programmazione ad alto livello, interpretato e multi-paradigma. A differenza dei linguaggi puramente compilati (come il C), Python utilizza un sistema ibrido che garantisce portabilità e flessibilità.

2 Il Flusso di Esecuzione

Il passaggio dal codice sorgente all'esecuzione sulla CPU non è diretto, ma attraversa diverse fasi cruciali:

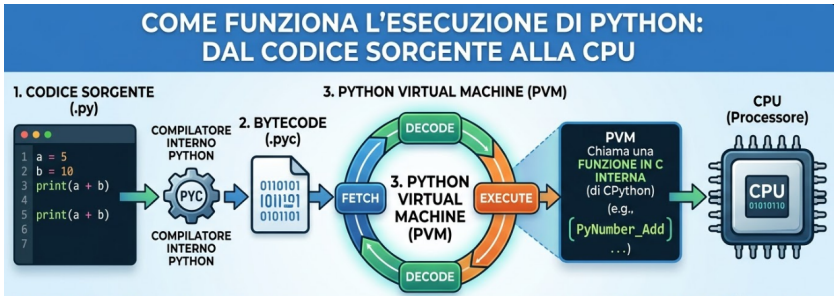


Figura 1: Funzionamento python con PVM

- **Compilazione in Bytecode** Quando un file `.py` viene eseguito, l'interprete (solitamente CPython) lo traduce in un formato intermedio chiamato **Bytecode**. Questo codice non è leggibile dall'uomo né direttamente dalla CPU, ma è ottimizzato per l'interprete. I file risultanti hanno spesso estensione `.pyc`.

- **La python Virtual Machine (PVM)** La PVM è il cuore del runtime di Python. Si tratta di un software che emula un microprocessore e agisce come strato di astrazione tra il Bytecode e l'hardware sottostante.

- **Il Meccanismo Interno: L'Interpreter Loop** La PVM opera attraverso un ciclo continuo di *Fetch-Decode-Execute*. Il dettaglio del funzionamento è il seguente:

  1. **Fetch:** La PVM preleva la successiva istruzione di Bytecode.
  2. **Decode:** L'istruzione viene identificata (es. `BINARY_ADD`).
  3. **Execute:** La PVM richiama una funzione interna scritta in **linguaggio C** (le cosiddette *C-API* di CPython).
  4. **Linguaggio Macchina:** La funzione C, essendo già compilata, viene eseguita direttamente dal processore come sequenza di istruzioni binarie.

2.1 Confronto tra C e Python

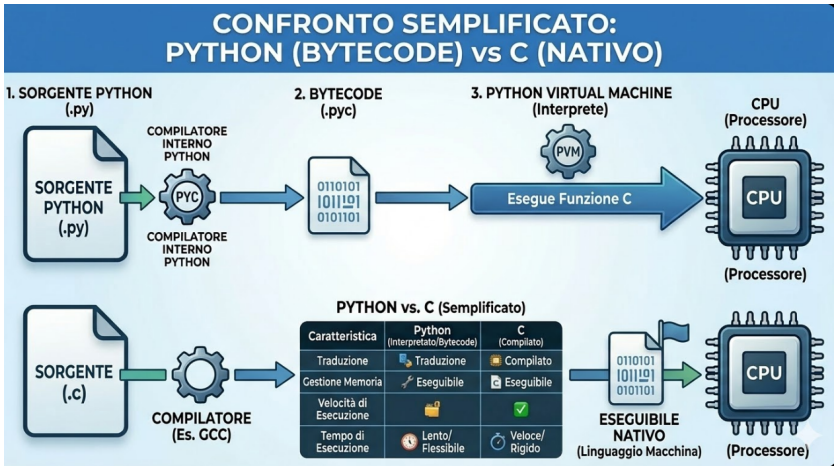


Figura 2: Confronto tra C e Python

Caratteristica	Linguaggio C	Python
Traduzione	Compilazione statica	Compilazione JIT/Bytecode
Eseguibile	Binario nativo (OS-specific)	Bytecode (Universale)
Astrazione	Bassa (vicino all'hardware)	Alta (astrazione totale)
Gestione Memoria	Manuale (`malloc/free`)	Automatica (Garbage Collector)

Tabella 1: Differenze architetturali principali.

2.2 Esempio

Supponiamo di avere il seguente codice python:

```
Week1 > utility.py > ...
1  def sum(a, b):
2      print('ciao')
3      return a + b
4
5
6  import dis      # Disassembler of Python byte code into mnemonics.
7
8  dis.dis(sum)    # dis.dis(...) Disassemble classes, methods, functions, and other compiled objects.
9                  # In this case, the sum function
10
```

Il modulo *dis* è il disassemblatore. Il suo compito è prendere il codice Python (che è scritto in un linguaggio comprensibile agli umani) e mostrarti le istruzioni a basso livello che la Python Virtual Machine (PVM) esegue effettivamente "sotto il cofano".

L'output è il seguente:

```
PS C:\Users\miche\Desktop\GitRepositories\DesignAnalysisOfAlgorithms\Week1> python utility.py
  2           0 LOAD_GLOBAL              0 (print)
              2 LOAD_CONST               1 ('ciao')
              4 CALL_FUNCTION            1
              6 POP_TOP

  3           8 LOAD_FAST                0 (a)
             10 LOAD_FAST                1 (b)
             12 BINARY_ADD
             14 RETURN_VALUE
```

Il blocco `print('ciao')` (Righe 0-6) Qui Python sta preparando una chiamata a funzione:

- **LOAD_GLOBAL 0 (print)**: Python deve andare a cercare la funzione print. Dato che non è definita dentro la tua funzione sum, la cerca nello spazio "Globale" (o nei built-ins).
- **LOAD_CONST 1 ('ciao')**: Carica l'argomento da passare alla funzione.
- **CALL_FUNCTION 1: Dice alla PVM**: "*Prendi la funzione che hai caricato, passale 1 argomento e avviala*". Qui la PVM sospende per un attimo sum e va a eseguire il codice C di print.

- **POP_TOP**: Questa è fondamentale. La funzione print restituisce sempre None, che viene messo sullo stack. Dato che non stai salvando il risultato di print in una variabile (es. non hai scritto x = print('ciao')), Python "pulisce" lo stack eliminando quel None.

Il blocco `return a + b` (Righe 8-14)

- **LOAD_FAST 0 (a) e 1 (b)**: Python carica i valori dei parametri locali. L'uso di *FAST* indica che l'accesso avviene tramite un indice in un array locale, rendendolo molto più rapido rispetto alla ricerca in un dizionario globale.

- **BINARY_ADD**: L'istruzione preleva i due valori dallo stack e invoca la routine C corrispondente per eseguire l'addizione. Il risultato viene poi riposto in cima allo stack.

- **RETURN_VALUE**: Conclude l'esecuzione della funzione restituendo al chiamante l'elemento attualmente presente in cima allo stack.

2.3 Conclusione

Sebbene l'interposizione della PVM renda Python intrinsecamente più lento del C, questo approccio permette una gestione della memoria semplificata, una portabilità cross-platform immediata e una velocità di sviluppo superiore, delegando le operazioni critiche a routine interne pre-ottimizzate in C.

3 Trees Hierarchy

As a preview of the remainder of this chapter, Figure 8.5 portrays our own hierarchy of classes for representing various forms of a tree.

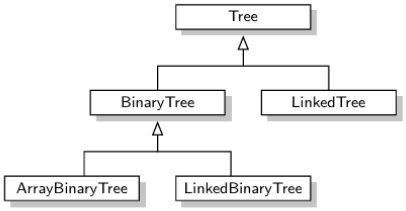
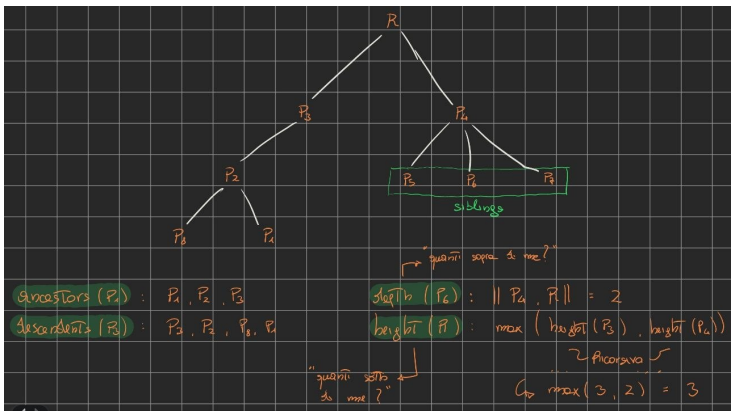


Figure 8.5: Our own inheritance hierarchy for modeling various abstractions and implementations of tree data structures. In the remainder of this chapter, we provide implementations of Tree, BinaryTree, and LinkedBinaryTree classes, and high-level sketches for how LinkedTree and ArrayBinaryTree might be designed.

3.1 General Tree



- Ogni nodo, escluso il nodo in alto (Radice), ha un padre e un numero arbitrario di figli.
- Due nodi che sono figli dello stesso padre sono frtelli;
- Un nodo è una foglia se non ha figli.
- Un nodo è interno se ha almeno un figlio.
- un nodo v è un **antenato** di un nodo u se $u = v$ o u è un antenato del padre di v.
- Un nodo v è un **descendente** di un nodo u se u è un antenato di v.

- Un **edge** è una coppia di nodi *(u, v)* tale che u è il padre di v o viceversa.
- Un **path** è una sequenza di nodi tali che qualsiasi coppia di nodi consecutivi è un **edge**.

3.1.1 General Tree - Abstract Data Type

structure. A position object for a tree supports the method:

p.element(): Return the element stored at position p.

The tree ADT then supports the following ***accessor methods***, allowing a user to navigate the various positions of a tree:

T.root(): Return the position of the root of tree T, or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p, or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that are contained in tree T.

T.is_empty(): Return True if tree T does not contain any positions.

T.positions(): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

3.2 Binary Tree

8.2 Binary Trees

A ***binary tree*** is an ordered tree with the following properties:

1. Every node has at most two children.
2. Each child node is labeled as being either a ***left child*** or a ***right child***.
3. A left child precedes a right child in the order of children of a node.

The subtree rooted at a left or right child of an internal node v is called a ***left subtree*** or ***right subtree***, respectively, of v. A binary tree is ***proper*** if each node has either zero or two children. Some people also refer to such trees as being ***full*** binary trees. Thus, in a proper binary tree, every internal node has exactly two children. A binary tree that is not proper is ***improper***.

Figura 3: Binary Tree

3.2.1 Binary Tree - Abstract Data Type

8.2.1 The Binary Tree Abstract Data Type

As an abstract data type, a binary tree is a specialization of a tree that supports three additional accessor methods:

T.left(p): Return the position that represents the left child of p, or None if p has no left child.

T.right(p): Return the position that represents the right child of p, or None if p has no right child.

T.sibling(p): Return the position that represents the sibling of p, or None if p has no sibling.

Just as in Section 8.1.2 for the tree ADT, we do not define specialized update methods for binary trees here. Instead, we will consider some possible update methods when we describe specific implementations and applications of binary trees.

Figura 4: Binary Tree ADT

3.2.2 Properties of Binary Trees

8.2.2 Properties of Binary Trees

Binary trees have several interesting properties dealing with relationships between their heights and number of nodes. We denote the set of all nodes of a tree T at the same depth d as ***level*** d of T. In a binary tree, level 0 has at most one node (the root), level 1 has at most two nodes (the children of the root), level 2 has at most four nodes, and so on. (See Figure 8.9.) In general, level d has at most 2^d nodes.

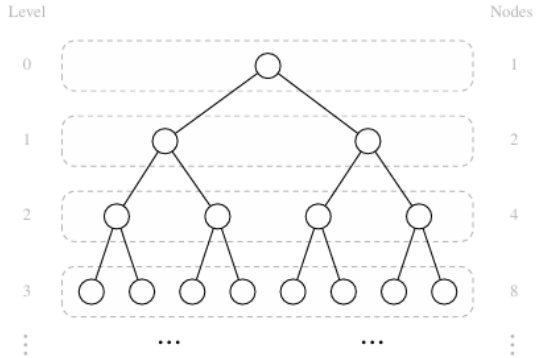


Figure 8.9: Maximum number of nodes in the levels of a binary tree.

We can see that the maximum number of nodes on the levels of a binary tree grows exponentially as we go down the tree. From this simple observation, we can derive the following properties relating the height of a binary tree T with its number of nodes. A detailed justification of these properties is left as Exercise R-8.8.

Proposition 8.8: *Let T be a nonempty binary tree, and let n, n_E, n_I and h denote the number of nodes, number of external nodes, number of internal nodes, and height of T, respectively. Then T has the following properties:*

1. $h+1 \leq n \leq 2^{h+1}-1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n+1) - 1 \leq h \leq n-1$

Also, if T is proper, then T has the following properties:

1. $2h+1 \leq n \leq 2^{h+1}-1$
2. $h+1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n+1) - 1 \leq h \leq (n-1)/2$

Figura 5: Binary Tree Properties

Relating Internal Nodes to External Nodes in a Proper Binary Tree

In addition to the earlier binary tree properties, the following relationship exists between the number of internal nodes and external nodes in a proper binary tree.

Proposition 8.9: *In a nonempty proper binary tree T, with n_E external nodes and n_I internal nodes, we have $n_E = n_I + 1$.*

Justification: We justify this proposition by removing nodes from T and dividing them up into two "piles," an internal-node pile and an external-node pile, until T becomes empty. The piles are initially empty. By the end, we will show that the external-node pile has one more node than the internal-node pile. We consider two cases:

Case 1: If T has only one node v, we remove v and place it on the external-node pile. Thus, the external-node pile has one node and the internal-node pile is empty.

Case 2: Otherwise (T has more than one node), we remove from T an (arbitrary) external node w and its parent v, which is an internal node. We place w on the external-node pile and v on the internal-node pile. If v has a parent u, then we reconnect u with the former sibling z of w, as shown in Figure 8.10. This operation, removes one internal node and one external node, and leaves the tree being a proper binary tree.

Repeating this operation, we eventually are left with a final tree consisting of a single node. Note that the same number of external and internal nodes have been removed and placed on their respective piles by the sequence of operations leading to this final tree. Now, we remove the node of the final tree and we place it on the external-node pile. Thus, the the external-node pile has one more node than the internal-node pile.

■

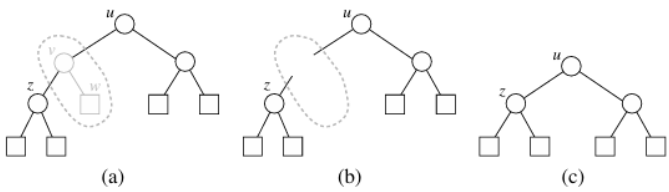


Figure 8.10: Operation that removes an external node and its parent node, used in the justification of Proposition 8.9.

Note that the above relationship does not hold, in general, for improper binary trees and nonbinary trees, although there are other interesting relationships that do hold. (See Exercises C-8.32 through C-8.34.)

Figura 6: Proepr Binary Tree Properties

3.3 General Tree Traversal

3.3.1 Preorder e Postorder

In a ***preorder traversal*** of a tree T, the root of T is visited first and then the subtrees rooted at its children are traversed recursively. If the tree is ordered, then the subtrees are traversed according to the order of the children. The pseudo-code for the preorder traversal of the subtree rooted at a position p is shown in Code Fragment 8.12.

```
Algorithm preorder(T, p):
  perform the "visit" action for position p
  for each child c in T.children(p) do
    preorder(T, c)                    {recursively traverse the subtree rooted at c}
```

Code Fragment 8.12: Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree T.

Figure 8.15 portrays the order in which positions of a sample tree are visited during an application of the preorder traversal algorithm.

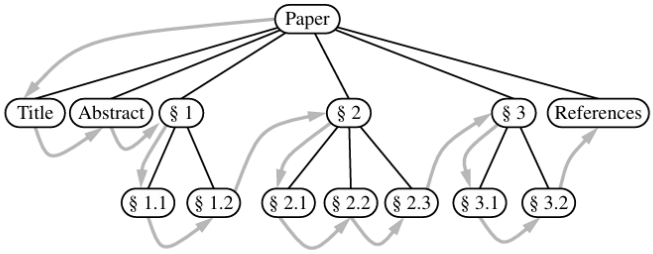


Figure 8.15: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

Figura 7: Preorder General Tree Traversal

Postorder Traversal

Another important tree traversal algorithm is the ***postorder traversal***. In some sense, this algorithm can be viewed as the opposite of the preorder traversal, because it recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name "postorder"). Pseudo-code for the postorder traversal is given in Code Fragment 8.13, and an example of a postorder traversal is portrayed in Figure 8.16.

```
Algorithm postorder(T, p):
  for each child c in T.children(p) do
    postorder(T, c)                {recursively traverse the subtree rooted at c}
  perform the "visit" action for position p
```

Code Fragment 8.13: Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree T.

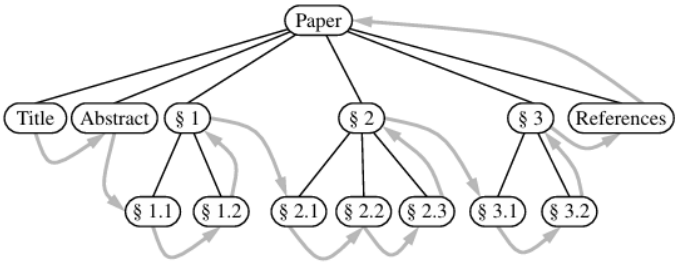


Figure 8.16: Postorder traversal of the ordered tree of Figure 8.15.

Figura 8: Postorder General Tree Traversal

Running Time Analysis Runtime Analysis di Preorder e Postorder General Tree Traversal

Both preorder and postorder traversal algorithms are efficient ways to access all the positions of a tree. The analysis of either of these traversal algorithms is similar to that of algorithm height2, given in Code Fragment 8.5 of Section 8.1.3. At each position p, the nonrecursive part of the traversal algorithm requires time $O(c_p + 1)$, where c_p is the number of children of p, under the assumption that the "visit" itself takes $O(1)$ time. By Proposition 8.5, the overall running time for the traversal of tree T is $O(n)$, where n is the number of positions in the tree. This running time is asymptotically optimal since the traversal must visit all the n positions of the tree.

Figura 9: Running Time Analysis di Postorder e Preorder General Tree Traversal

3.3.2 Breadth-First Tree Traversal

Although the preorder and postorder traversals are common ways of visiting the positions of a tree, another common approach is to traverse a tree so that we visit all the positions at depth d before we visit the positions at depth $d+1$. Such an algorithm is known as a ***breadth-first traversal***.

A breadth-first traversal is a common approach used in software for playing games. A ***game tree*** represents the possible choices of moves that might be made by a player (or computer) during a game, with the root of the tree being the initial configuration for the game. For example, Figure 8.17 displays a partial game tree for Tic-Tac-Toe.

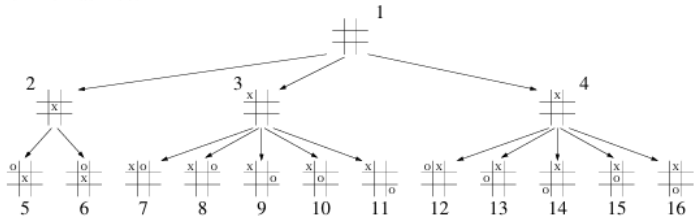


Figure 8.17: Partial game tree for Tic-Tac-Toe, with annotations displaying the order in which positions are visited in a breadth-first traversal.

A breadth-first traversal of such a game tree is often performed because a computer may be unable to explore a complete game tree in a limited amount of time. So the computer will consider all moves, then responses to those moves, going as deep as computational time allows.

Pseudo-code for a breadth-first traversal is given in Code Fragment 8.14. The process is not recursive, since we are not traversing entire subtrees at once. We use a queue to produce a FIFO (i.e., first-in first-out) semantics for the order in which we visit nodes. The overall running time is $O(n)$, due to the n calls to enqueue and n calls to dequeue.

```
Algorithm breadthfirst(T):
  Initialize queue Q to contain T.root()
  while Q not empty do
    p = Q.dequeue()                    {p is the oldest entry in the queue}
    perform the "visit" action for position p
    for each child c in T.children(p) do
      Q.enqueue(c)     {add p's children to the end of the queue for later visits}
```

Code Fragment 8.14: Algorithm for performing a breadth-first traversal of a tree.

Figura 10: Breadth-First General Tree Traversal

3.3.3 Inorder Traversal

The standard preorder, postorder, and breadth-first traversals that were introduced for general trees, can be directly applied to binary trees. In this section, we introduce another common traversal algorithm specifically for a binary tree.

During an ***inorder traversal***, we visit a position between the recursive traversals of its left and right subtrees. The inorder traversal of a binary tree T can be informally viewed as visiting the nodes of T "from left to right." Indeed, for every position p, the inorder traversal visits p after all the positions in the left subtree of p and before all the positions in the right subtree of p. Pseudo-code for the inorder traversal algorithm is given in Code Fragment 8.15, and an example of an inorder traversal is portrayed in Figure 8.18.

```
Algorithm inorder(p):
  if p has a left child lc then
    inorder(lc)                          {recursively traverse the left subtree of p}
  perform the "visit" action for position p
  if p has a right child rc then
    inorder(rc)                          {recursively traverse the right subtree of p}
```

Code Fragment 8.15: Algorithm inorder for performing an inorder traversal of a subtree rooted at position p of a binary tree.

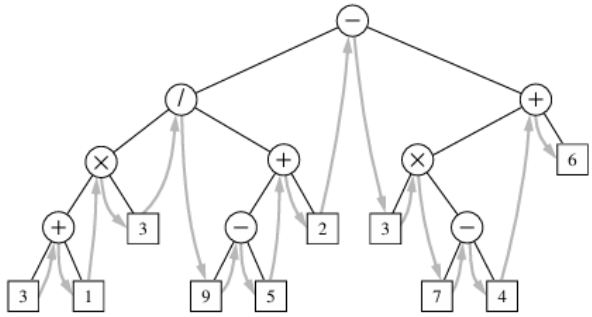


Figure 8.18: Inorder traversal of a binary tree.

The inorder traversal algorithm has several important applications. When using a binary tree to represent an arithmetic expression, as in Figure 8.18, the inorder traversal visits positions in a consistent order with the standard representation of the expression, as in $3+1\times 3/9-5+2\ldots$ (albeit without parentheses).

Figura 11: Inorder General Tree Traversal

3.3.4 Tree Traversal in Python

When first defining the tree ADT in Section 8.1.2, we stated that tree T should include support for the following methods:

T.positions( ): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

At that time, we did not make any assumption about the order in which these iterations report their results. In this section, we demonstrate how any of the tree traversal algorithms we have introduced could be used to produce these iterations.

To begin, we note that it is easy to produce an iteration of all elements of a tree, if we rely on a presumed iteration of all positions. Therefore, support for the iter(T) syntax can be formally provided by a concrete implementation of the special method __iter__ within the abstract base class Tree. We rely on Python's generator syntax as the mechanism for producing iterations. (See Section 1.8.) Our implementation of Tree.__iter__ is given in Code Fragment 8.16.

```
def __iter__(self):
  """Generate an iteration of the tree's elements."""
  for p in self.positions( ):          # use same order as positions()
    yield p.element( )                 # but yield each element
```

Code Fragment 8.16: Iterating all elements of a Tree instance, based upon an iteration of the positions of the tree. This code should be included in the body of the Tree class.

To implement the positions method, we have a choice of tree traversal algorithms. Given that there are advantages to each of those traversal orders, we will provide independent implementations of each strategy that can be called directly by a user of our class. We can then trivially adapt one of those as a default order for the positions method of the tree ADT.

Figura 12: Tree Traversal in Python

Preorder in Python Preorder in Python

We begin by considering the ***preorder traversal*** algorithm. We will support a public method with calling signature T.preorder( ) for tree T, which generates a preorder iteration of all positions within the tree. However, the recursive algorithm for generating a preorder traversal, as originally described in Code Fragment 8.12, must be parameterized by a specific position within the tree that serves as the root of a subtree to traverse. A standard solution for such a circumstance is to define a nonpublic utility method with the desired recursive parameterization, and then to have the public method preorder invoke the nonpublic method upon the root of the tree. Our implementation of such a design is given in Code Fragment 8.17.


```
def preorder(self):
  """Generate a preorder iteration of positions in the tree."""
  if not self.is_empty( ):
    for p in self._subtree_preorder(self.root( )):        # start recursion
      yield p

def _subtree_preorder(self, p):
  """Generate a preorder iteration of positions in subtree rooted at p."""
  yield p                                          # visit p before its subtrees
  for c in self.children(p):                       # for each child c
    for other in self._subtree_preorder(c):        # do preorder of c's subtree
      yield other                                  # yielding each to our caller
```

Code Fragment 8.17: Support for performing a preorder traversal of a tree. This code should be included in the body of the Tree class.

Formally, both preorder and the utility _subtree_preorder are generators. Rather than perform a "visit" action from within this code, we yield each position to the caller and let the caller decide what action to perform at that position.

The _subtree_preorder method is the recursive one. However, because we are relying on generators rather than traditional functions, the recursion has a slightly different form. In order to yield all positions within the subtree of child c, we loop over the positions yielded by the recursive call self._subtree_preorder(c), and re-yield each position in the outer context. Note that if p is a leaf, the for loop over self.children(p) is trivial (this is the base case for our recursion).

We rely on a similar technique in the public preorder method to re-yield all positions that are generated by the recursive process starting at the root of the tree; if the tree is empty, nothing is yielded. At this point, we have provided full support for the preorder generator. A user of the class can therefore write code such as

```
for p in T.preorder( ):
  # "visit" position p
```

The official tree ADT requires that all trees support a positions method as well. To use a preorder traversal as the default order of iteration, we include the definition shown in Code Fragment 8.18 within our Tree class. Rather than loop over the results returned by the preorder call, we return the entire iteration as an object.

```
def positions(self):
  """Generate an iteration of the tree's positions."""
  return self.preorder( )              # return entire preorder iteration
```

Code Fragment 8.18: An implementation of the positions method for the Tree class that relies on a preorder traversal to generate the results.

Figura 13: Preorder in Python

Postorder in Python Postorder in Python

Postorder Traversal

We can implement a postorder traversal using very similar technique as with a preorder traversal. The only difference is that within the recursive utility for a postorder we wait to yield position p until *after* we have recursively yield the positions in its subtrees. An implementation is given in Code Fragment 8.19.

```
def postorder(self):
  """Generate a postorder iteration of positions in the tree."""
  if not self.is_empty():
    for p in self._subtree_postorder(self.root()):        # start recursion
      yield p

def _subtree_postorder(self, p):
  """Generate a postorder iteration of positions in subtree rooted at p."""
  for c in self.children(p):                    # for each child c
    for other in self._subtree_postorder(c):    # do postorder of c's subtree
      yield other                               # yielding each to our caller
  yield p                                       # visit p after its subtrees
```

Code Fragment 8.19: Support for performing a postorder traversal of a tree. This code should be included in the body of the Tree class.

Figura 14: Postorder in Python

Breadth-First in Python Breadth-First in Python

Breadth-First Traversal

In Code Fragment 8.20, we provide an implementation of the breadth-first traversal algorithm in the context of our Tree class. Recall that the breadth-first traversal algorithm is not recursive; it relies on a queue of positions to manage the traversal process. Our implementation uses the LinkedQueue class from Section 7.1.2, although any implementation of the queue ADT would suffice.

Figura 15: Breadth-First in Python

Inoredr in Python Inorder in Python

Inorder Traversal for Binary Trees

The preorder, postorder, and breadth-first traversal algorithms are applicable to all trees, and so we include their implementations within the Tree abstract base class. Those methods are inherited by the abstract BinaryTree class, the concrete LinkedBinaryTree class, and any other dependent tree classes we might develop.

The inorder traversal algorithm, because it explicitly relies on the notion of a left and right child of a node, only applies to binary trees. We therefore include its definition within the body of the BinaryTree class. We use a similar technique to implement an inorder traversal (Code Fragment 8.21) as we did with preorder and postorder traversals.


```
def breadthfirst(self):
  """Generate a breadth-first iteration of the positions of the tree."""
  if not self.is_empty( ):
    fringe = LinkedQueue( )              # known positions not yet yielded
    fringe.enqueue(self.root( ))         # starting with the root
    while not fringe.is_empty( ):
      p = fringe.dequeue( )              # remove from front of the queue
      yield p                            # report this position
      for c in self.children(p):
        fringe.enqueue(c)                # add children to back of queue
```

Code Fragment 8.20: An implementation of a breadth-first traversal of a tree. This code should be included in the body of the Tree class.

```
def inorder(self):
  """Generate an inorder iteration of positions in the tree."""
  if not self.is_empty( ):
    for p in self._subtree_inorder(self.root( )):
      yield p

def _subtree_inorder(self, p):
  """Generate an inorder iteration of positions in subtree rooted at p."""
  if self.left(p) is not None:        # if left child exists, traverse its subtree
    for other in self._subtree_inorder(self.left(p)):
      yield other
  yield p                             # visit p between its subtrees
  if self.right(p) is not None:       # if right child exists, traverse its subtree
    for other in self._subtree_inorder(self.right(p)):
      yield other
```

Code Fragment 8.21: Support for performing an inorder traversal of a binary tree. This code should be included in the BinaryTree class (given in Code Fragment 8.7).

For many applications of binary trees, an inorder traversal provides a natural iteration. We could make it the default for the BinaryTree class by overriding the positions method that was inherited from the Tree class (see Code Fragment 8.22).

```
# override inherited version to make inorder the default
def positions(self):
  """Generate an iteration of the tree's positions."""
  return self.inorder( )              # make inorder the default
```

Code Fragment 8.22: Defining the BinaryTree.position method so that positions are reported using inorder traversal.

Figura 16: Inorder in Python